NP1: Definitions

NP: Overview

1. How do prove that a problem is computationally difficult?
   - Hard to devise an efficient algorithm to solve it for all inputs
2. Outline:
   - NP
   - NP-Complete Reductions
   - NP Completeness
     - 3SAT
     - Graph problems
     - Knapsack
     - Halting problem (undecidable)

Lecture Outline

1. Computational Complexity
   - What does NP-completeness mean?
   - What does P=NP or P!=NP mean?
   - How do we show that a problem is intractable?
     - Intractable: Unlikely to be solved efficiently

Complexity Classes

1. NP: Class of all search problems
   - Can use decision problems instead of search problems
     - Search problems remove the need for witnesses in specific instances
   - Search problems
     - Rough definition: Problem where we can efficiently verify solutions
       * Efficiently: In polynomial time
2. P: Class of search problems that are solvable in polynomial time
   - P is a subset of NP

Comparing P and NP

1. P ?= NP
   - P = NP: It's as difficult to solve a problem as it is to check if a solution is correct
   - P != NP: It seems much easier to check if a solution is correct than to generate a solution

Search Problems

1. Search problem:
   - Form: Given instance I
     - Find a solution S for I if one exists
     - Output NO if I has no solutions
   - Requirement: To be a search problem:
     - If given an instance I and a solution S, then we can verify that S is a solution to I in polynomial time (polynomial in |I|)
     - Show an algorithm that can verify a solution in polynomial time

SAT Problem

1. SAT
   - Input: Boolean formula f in CNF with n variables and m clauses

- Output: Satisfying assignment if one exists, NO otherwise

SAT Example

1. f = (x_3 v !x_2 v !x_1) ^ (x_1) ^ (x_2 v !x_3) ^ (!x_1 v !x_3)
   - x_1 = TRUE
   - x_2 = FALSE
   - x_3 = FALSE

SAT in NP

1. Given f and assignment of TRUE or FALSE of each variable, what is the running time to verify?
   - O(nm)
2. Prove SAT is in NP:
   - SAT is a search problem of the correct form
   - Solutions can be verified in O(nm) time, so a solution is verifiable in polynomial time

Colorings in NP

1. k-colorings problem:
   - Input: Undirected G=(V,E) and integer k > 0
   - Output: Assign each vertex a color in {1,2,...k} so that adjacent vertices get different colors and NO if no such k-coloring exists for G
2. k-colorings in NP:
   - k-colorings is a search problem
   - Given G and a coloring, we can check that (v,w) in E, color of v is different from color of w in O(m) time

MST

1. MST problem:
   - Input: G=(V,E) with positive edge lengths
   - Output: Tree T with minimum weight
2. MST in NP?
   - TRUE
3. MST in P?
   - TRUE

MST in NP

1. MST in NP?
   - Given G and T:
     - Run BFS/DFS to check that T is a tree
     - Run Kruskal's or Prim's to check that T has minimum weight
   - Total time of verification algorithm is O(mlog(n)).

MST in P

1. MST in P?
   - MST is a search problem and we can find a solution in polynomial time
     - Kruskal's and Prim's algorithms are both O(mlog(n))

Knapsack Problem

1. Knapsack in NP?

- Input: N objects with integer weights $w_1, \ldots, w_n$ and integer values $v_1, \ldots, v_n$ and a total capacity B
- Output: Subset S of objects with:
  - Total weight <= B
  - Maximum total value
- With or without repetition

Knapsack Complexity

1. Knapsack in NP?
   - FALSE
     - Can't verify the total value is a maximum in polynomial time; the brute force verification solution is exponential, $O(2^n)$, while dynamic programming solution is O(nB) (still not polynomial)
     - Can verify the weight in polynomial time; O(n)
     - Knapsack is not known to be in NP
       - Can't prove that knapsack is in NP and also can't prove that knapsack is not in NP
2. Knapsack in P?
   - FALSE
     - Knapsack not known to be in NP
     - Known solutions are pseudo-polynomial

Knapsack Search

1. Knapsack-search
   - Input: N objects with integer weights $w_1, \ldots, w_n$ and integer values $v_1, \ldots, v_n$ and a total capacity B and goal g
   - Output: Subset S with weight <= B and value >= g and NO if no such S exists
2. Can do binary search over the g parameter and use that to find the maximum
   - Search up to and including the maximum value
   - This would result in a log(V) solution, which we could use to verify our Knapsack solution in polynomial time

Knapsack Search in NP

1. Knapsack-search exists in NP
   - Given input and solution S, need to check in polynomial-time:
     - Total weight <= B takes O(nlog(W)
     - Total value >= g takes O(nlog(V)
   - Both are polynomial
2. Knapsack-search is in NP

NP Acronym For?

1. Terminology
   - P = Polynomial time: Class of search problems that can be solved in polynomial time
   - NP = Non-deterministic polynomial time: Problems that can be solved in polynomial-time on a non-deterministic machine
     - Non-deterministic: Allowed to guess at each step
     - There is a choice of branchings that lead to an accepting state

P vs NP

1. P vs NP
   - NP: All search problems

- P: Search problems that can be solved in polynomial-time

NP-Completeness

1. If P != NP
   - NP-complete are the intractable problems in NP
     - Hardest problems in the class NP
     - In NP, not in P
   - If P != NP, then all NP-complete problems are not in P
     - If an NP-complete problem can be solved in polynomial-time, then all problems in NP can be solved in polynomial-time
   - Show each search problem can be reduced to SAT
     - Then, if we can solve SAT in polynomial-time, we can solve all problems that reduce to SAT in polynomial-time

P vs NP

SAT is NP-Complete

1. SAT is NP-complete means:
   - SAT exists in NP
     - We have shown we can efficiently verify solutions to SAT
     - If we can solve SAT in polynomial-time, then we can solve every problem in NP in polynomial time
       * Traveling salesman, independent set, colorings, MST
2. If P != NP, then SAT is not in P
   - Formalize the notion of a reduction

Reductions

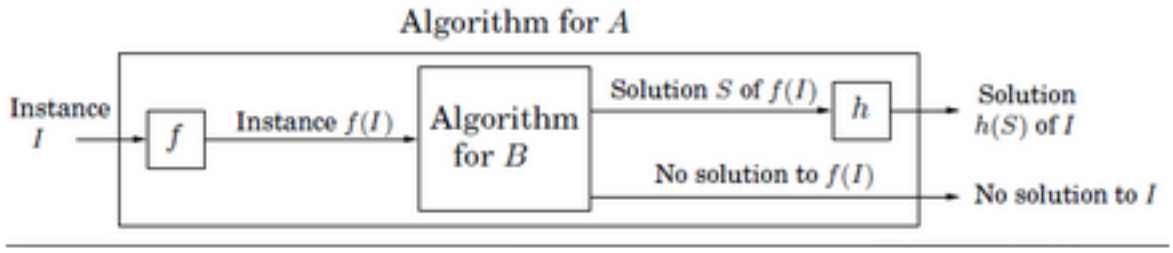
1. Problems A & B (example: A = colorings, B = SAT)
   - A -> B or A <= B
   - Reducing A to B
     - If we can solve problem B in polynomial-time, then we can use that algorihtm to solve A in polynomial-time

How to do a Reduction

1. Colorings -> SAT
   - Suppose there is a polynomial-time algorithm for SAT and use it to get a polynomial-time algorithm for colorings
2. Process

- Transform input for colorings problem to input for SAT problem
- Put input into SAT algorithm
- Define a transformation for output of algorithm to desired output

---



Reduction

More on Reductions

1. Colorings -> SAT:
   - We need to define f and h
     - f: Input for colorings G, k -> input for SAT f(G, k)
     - h: Solution for f(G, k) -> solution for colorings h(s)
   - Need to prove: S is a solution to f(G,k) iff h(s) is a solution to (G,k)

NP-Completeness Proof

1. To show: Independent sets (IS) is NP-complete
   - Need to show:
     - IS is in NP (verify solutions in polynomial-time)
     - For all A in NP, A -> IS (A reduces to IS)

Simpler Proof Approach

1. Suppose we know SAT is NP-complete
   - For all A in NP, A -> SAT
   - Suppose we show SAT -> IS
     - Then: A -> SAT -> IS
2. To show: Independent sets (IS) is NP-complete
   - Need to show:
     - IS is in NP (verify solutions in polynomial-time)
     - SAT -> IS
       * A -> SAT -> IS

Practice Problems

1. P, NP, reductions
   - DPV 8.1: Optimization vs search
   - DPV 8.2: Search vs decision